because some components of the state may be inaccessible for observation, the sensors used for measuring them may be inaccurate, or the cost of measuring them more accurately may be prohibitive.

Often in such situations, the controller has access to only some of the components of the current state, and the corresponding observations may also be corrupted by stochastic uncertainty. For example in three-dimensional motion problems, the state may consist of the six-tuple of position and velocity components, but the observations may consist of noise-corrupted radar measurements of the three position components. This gives rise to problems of *partial* or *imperfect* state information, which have received a lot of attention in the optimization and artificial intelligence literature (see e.g., [Ber17a], [RuN16]; these problems are also popularly referred to with the acronym POMDP for *partially observed Markovian Decision problem*).

Generally, solving a POMDP exactly is typically intractable, even though there are DP algorithms for doing so. Thus in practice, POMDP are solved approximately, except under very special circumstances.

Despite their inherent computational difficulty, it turns out that conceptually, partial state information problems are no different than the perfect state information problems we have been addressing so far. In fact by various reformulations, we can reduce a partial state information problem to one with perfect state information, which involves a different and more complicated state, called a *sufficient statistic*. Once this is done, we can state an exact DP algorithm that is defined over the space of the sufficient statistic. Roughly speaking, a sufficient statistic is a quantity that summarizes the content of the information available up to k for the purposes of optimal control. This statement can be made more precise, but we will not elaborate further in this book; see e.g., the DP textbook [Ber17a].

A common sufficient statistic is the *belief state*, which we will denote by b_k. It is the probability distribution of x_k given all the observations that have been obtained by the controller and all the controls applied by the controller up to time k, and it can serve as "state" in an appropriate DP algorithm. The belief state can in principle be computed and updated by a variety of methods that are based on Bayes' rule, such as *Kalman filtering* (see e.g., [AnM79], [KuV86], [Kri16], [ChC17]) and *particle filtering* (see e.g., [GSS93], [DoJ09], [Can16], [Kri16]).

Example 1.6.4 (Bidirectional Parking)

Let us consider a more complex version of the parking problem of Example 1.6.1. As in that example, a driver is looking for inexpensive parking on the way to his destination, along a line of N parking spaces with a garage at the end. The difference is that the driver can move in either direction, rather than just forward towards the garage. In particular, at space i, the driver can park at cost $c(i)$ if i is free, can move to $i-1$ at a cost t_i^- or can move to $i+1$ at a cost t_i^+. Moreover, the driver records and remembers the free/taken

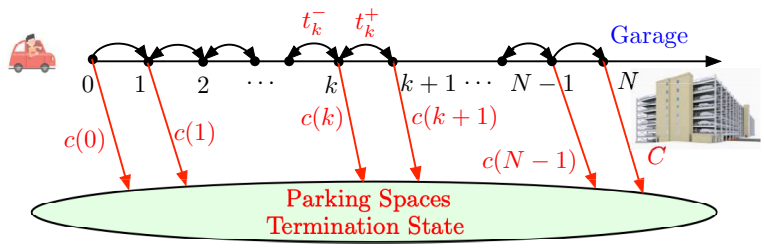

Figure 1.6.6 Cost structure and transitions of the bidirectional parking problem. The driver may park at space $k = 0, 1, \ldots, N-1$ at cost $c(k)$, if the space is free, can move to $k-1$ at cost t_k^- or can move to $k+1$ at cost t_k^+. At space N (the garage) the driver must park at cost C.

status of the spaces previously visited and may return to any of these spaces; see Fig. 1.6.6.

We assume that the probability $p(i)$ of a space i being free changes over time, i.e., a space found free (or taken) at a given visit may get taken (or become free, respectively) by the time of the next visit. The initial probabilities $p(i)$, before visiting any spaces, are known, and the mechanism by which these probabilities change over time is also known to the driver. As an example, we may assume that at each time stage, $p(i)$ increases by a certain known factor with some probability ξ and decreases by another known factor with the complementary probability $1-\xi$.

Here the belief state is the vector of current probabilities

$$\big(p(0), \ldots, p(N-1)\big),$$

and it can be updated with a simple algorithm at each time based on the new observation: the free/taken status of the space visited at that time.

We can use the belief state as the basis of an exact DP algorithm for computing an optimal policy. This algorithm is typically intractable computationally, but it is conceptually useful, and it can form the starting point for approximations. It has the form

$$J_k^*(b_k) = \min_{u_k \in U_k} \Big[\hat{g}_k(b_k, u_k) + E_{z_{k+1}} \Big\{ J_{k+1}^*\big(F_k(b_k, u_k, z_{k+1})\big) \mid b_k, u_k \Big\} \Big], \tag{1.73}$$

where:

$J_k^*(b_k)$ denotes the optimal cost-to-go starting from belief state b_k at stage k.

U_k is the control constraint set at time k (since the state x_k is unknown at stage k, U_k must be independent of x_k).

$\hat{g}_k(b_k, u_k)$ denotes the expected stage cost of stage k. It is calculated as the expected value of the stage cost $g_k(x_k, u_k, w_k)$, with the joint

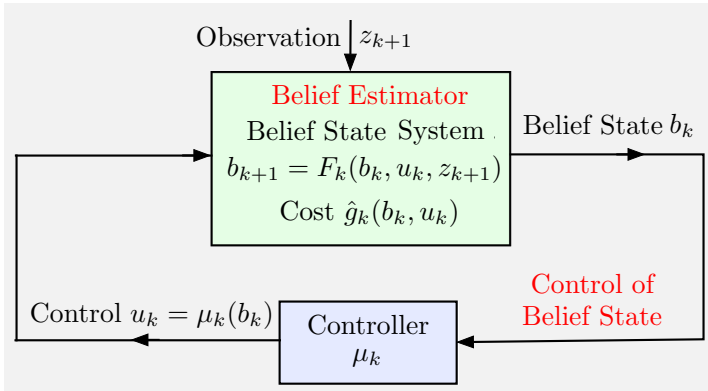

Figure 1.6.7 Schematic illustration of the view of an imperfect state information problem as one of perfect state information, whose state is the belief state b_k, i.e., the conditional probability distribution of x_k given all the observations up to time k. The observation z_{k+1} plays the role of the stochastic disturbance. The function F_k is a sequential estimator that updates the current belief state b_k.

distribution of (x_k, w_k) determined by the belief state b_k and the distribution of w_k.

$F_k(b_k, u_k, z_{k+1})$ denotes the belief state at the next stage, given that the current belief state is b_k, control u_k is applied, and observation z_{k+1} is received following the application of u_k:

$$b_{k+1} = F_k(b_k, u_k, z_{k+1}). \tag{1.74}$$

This is the system equation for a perfect state information problem with state b_k, control u_k, "disturbance" z_{k+1}, and cost per stage $\hat{g}_k(b_k, u_k)$. The function F_k is viewed as a sequential *belief estimator*, which updates the current belief state b_k based on the new observation z_{k+1}. It is given by either an explicit formula or an algorithm (such as Kalman filtering or particle filtering) that is based on the probability distribution of z_k and the use of Bayes' rule.

The expected value $E_{z_{k+1}}\{\cdot \mid b_k, u_k\}$ is taken with respect to the distribution of z_{k+1}, given b_k and u_k. Note that z_{k+1} is random, and its distribution depends on x_k and u_k, so the expected value

$$E_{z_{k+1}}\Big\{J^*_{k+1}\big(F_k(b_k, u_k, z_{k+1})\big) \mid b_k, u_k\Big\}$$

in Eq. (1.73) is a function of b_k and u_k.

The algorithm (1.73) is just the ordinary DP algorithm for the perfect state information problem shown in Fig. 1.6.7. It involves the system/belief estimator (1.74) and the cost per stage $\hat{g}_k(b_k, u_k)$. Note that since b_k takes

values in a continuous space, the algorithm (1.73) will typically require an approximate implementation, using approximation in value space methods.

We refer to the textbook [Ber17a], Chapter 4, for a detailed derivation of the DP algorithm (1.73), and to the monograph [BeS78] for a mathematical treatment that applies to infinite-dimensional state and disturbance spaces as well.

An Alternative DP Algorithm for POMDP

The DP algorithm (1.73) is not the only one that can be used for POMDP. There is also an exact DP algorithm that operates in the space of information vectors I_k, defined by

$$I_k = \{z_0, u_0, \ldots, z_{k-1}, u_{k-1}, z_k\},$$

where z_k is the observation received at time k. This is another sufficient statistic, and hence an alternative to the belief state b_k. In particular, we can view I_k as a state of the POMDP, which evolves over time according to the equation

$$I_{k+1} = (I_k, z_{k+1}, u_k).$$

Denoting by $J_k^*(I_k)$ the optimal cost starting at information vector I_k at time k, the DP algorithm takes the form

$$\begin{aligned} J_k^*(I_k) = \min_{u_k \in U_k(x_k)} E_{w_k, z_{k+1}} \Big\{ & g_k(x_k, u_k, w_k) + \\ & J_{k+1}^*\big(I_k, z_{k+1}, u_k\big) \mid I_k, u_k \Big\}, \end{aligned} \tag{1.75}$$

for $k = 0, \ldots, N-1$, with $J_N^*(I_N) = E\big\{g_N(x_N) \mid I_N\big\}$; see e.g., the DP textbook [Ber17a], Section 4.1.

A drawback of the preceding approach is that the information vector I_k is growing in size over time, thereby leading to a nonstationary system even in the case of an infinite horizon problem with a stationary system and cost function. This difficulty can be remedied in an approximation scheme that uses a finite history of the system (a fixed number of most recent observations) as state, thereby working effectively with a stationary finite-state system; see the paper by White and Scherrer [WhS94]. In particular, this approach is used in large language models such as ChatGPT and DeepSeek.

Finite-memory approximations for POMDP can be viewed within the context of feature-based approximation architectures, as we will discuss in Chapter 3 (see Example 3.1.6). Moreover, the finite-history scheme can be generalized through the concept of a *finite-state controller*; see the paper by Yu and Bertsekas [YuB08], which also addresses the issue of convergence of the approximation error to zero as the size of the finite-history or finite-state controller is increased.

1.6.7 Multiagent Problems and Multiagent Rollout

In this book, we will view a multiagent system as a collection of decision making entities, called *agents*, which aim to optimally achieve a common goal.† The agents accomplish this by collecting and exchanging information, and otherwise interacting with each other. The agents can be software programs or physical entities such as robots, and they may have different capabilities.

Among the generic challenges of efficient implementation of multiagent systems, one may note issues of limited communication and lack of fully shared information, due to factors such as limited bandwidth, noisy channels, and lack of synchronization. Another important generic issue is that as the number of agents increases, the size of the set of possible joint decisions of the agents increases exponentially, thereby complicating control selection by lookahead minimization. In this section, we will focus on ways to resolve this latter difficulty for problems where the agents fully share information, and in Section 2.9 we will address some of the challenges of problems where the agents may have some autonomy, and act without fully coordinating with each other.

For a mathematical formulation, let us consider the discounted infinite horizon problem and a special structure of the control space, whereby the control u consists of m components, $u = (u^1, \ldots, u^m)$, with a separable control constraint structure $u^\ell \in U^\ell(x)$, $\ell = 1, \ldots, m$. Thus the control constraint set is the Cartesian product

$$U(x) = U^1(x) \times \cdots \times U^m(x), \tag{1.76}$$

where the sets $U^\ell(x)$ are given. This structure arises in applications involving distributed decision making by multiple agents; see Fig. 1.6.8.

In particular, we will view each component u^ℓ, $\ell = 1, \ldots, m$, as being chosen from within $U^\ell(x)$ by a separate "agent" (a decision making entity). For the sake of the following discussion, we assume that each set $U^\ell(x)$ is finite. Then the one-step lookahead minimization of the standard rollout scheme with base policy μ is given by

$$\tilde{u} \in \arg\min_{u \in U(x)} E_w\Big\{g(x,u,w) + \alpha J_\mu\big(f(x,u,w)\big)\Big\}, \tag{1.77}$$

and involves as many as n^m Q-factors, where n is the maximum number of elements of the sets $U^\ell(x)$ [so that n^m is an upper bound to the number of controls in $U(x)$, in view of its Cartesian product structure (1.76)]. Thus the standard rollout algorithm requires an exponential [order $O(n^m)$] number of Q-factor computations per stage, which can be overwhelming even for moderate values of m.

† In a more general version of a multiagent system, which is outside our scope, the agents may have different goals, and act in their own self-interest.

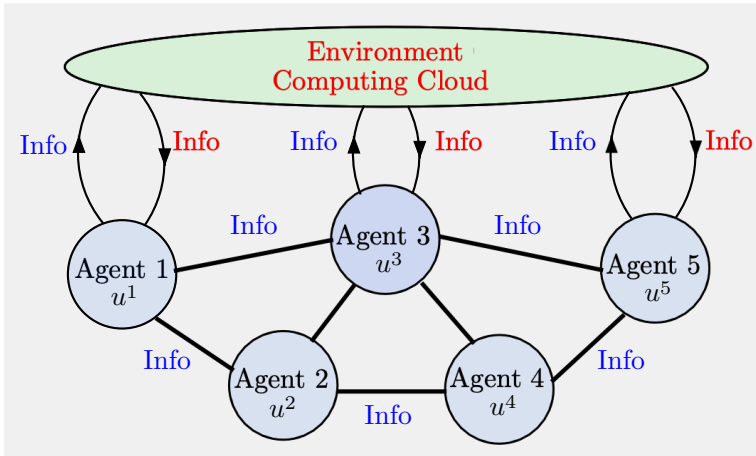

Figure 1.6.8 Schematic illustration of a multiagent problem. There are multiple "agents," and each agent $\ell = 1, \ldots, m$ controls its own decision variable u^ℓ. At each stage, agents exchange new information and also exchange information with the "environment," and then select their decision variables for the stage.

This potentially large computational overhead motivates a far more computationally efficient rollout algorithm, whereby the one-step lookahead minimization (1.77) is replaced by a sequence of m successive minimizations, *one-agent-at-a-time*, with the results incorporated into the subsequent minimizations. In particular, given a base policy $\mu = (\mu^1, \ldots, \mu^m)$, we perform at state x the sequence of minimizations

$$\tilde{\mu}^1(x) \in \arg \min_{u^1 \in U^1(x)} E_w \Big\{ g\big(x, u^1, \mu^2(x), \ldots, \mu^m(x), w\big) + \alpha J_\mu\big(f(x, u^1, \mu^2(x), \ldots, \mu^m(x), w)\big) \Big\},$$

$$\tilde{\mu}^2(x) \in \arg \min_{u^2 \in U^2(x)} E_w \Big\{ g\big(x, \tilde{\mu}^1(x), u^2, \mu^3(x) \ldots, \mu^m(x), w\big) + \alpha J_\mu\big(f(x, \tilde{\mu}^1(x), u^2, \mu^3(x), \ldots, \mu^m(x), w)\big) \Big\},$$

$$\ldots \qquad \ldots \qquad \ldots \qquad \ldots$$

$$\tilde{\mu}^m(x) \in \arg \min_{u^m \in U^m(x)} E_w \Big\{ g\big(x, \tilde{\mu}^1(x), \tilde{\mu}^2(x), \ldots, \tilde{\mu}^{m-1}(x), u^m, w\big) + \alpha J_\mu\big(f(x, \tilde{\mu}^1(x), \tilde{\mu}^2(x), \ldots, \tilde{\mu}^{m-1}(x), u^m, w)\big) \Big\}.$$

Thus each agent component u^ℓ is obtained by a minimization with the preceding agent components $u^1, \ldots, u^{\ell-1}$ fixed at the previously computed values of the rollout policy, and the following agent components $u^{\ell+1}, \ldots, u^m$ fixed at the values given by the base policy. This algorithm requires order

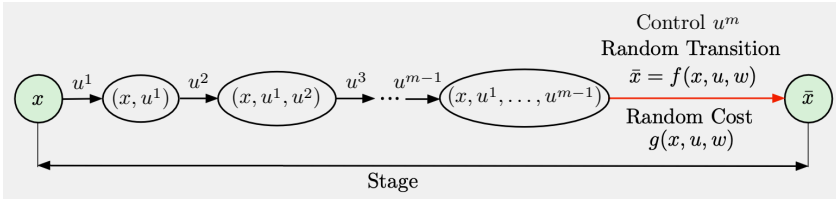

Figure 1.6.9 Equivalent formulation of the stochastic optimal control problem for the case where the control u consists of m components $u^1, u^2, \ldots, u^m$:

$$u = (u^1, \ldots, u^m) \in U(x) = U^1(x) \times \cdots \times U^m(x).$$

The figure depicts the kth stage transitions. Starting from state x, we generate the intermediate states

$$(x, u^1), (x, u^1, u^2), \ldots, (x, u^1, \ldots, u^{m-1}),$$

using the respective controls $u^1, \ldots, u^{m-1}$. The final control u^m leads from $(x, u^1, \ldots, u^{m-1})$ to $\bar{x} = f(x, u, w)$, and the random cost $g(x, u, w)$ is incurred.

$O(nm)$ Q-factor computations per stage, a potentially huge computational saving over the order $O(n^m)$ computations required by standard rollout.

A key idea here is that the computational requirements of the rollout one-step minimization (1.77) are proportional to the number of controls in the set $U(x_k)$ and are independent of the size of the state space. This motivates a reformulation of the problem, first suggested in the neuro-dynamic programming book [BeT96], Section 6.1.4, whereby *control space complexity is traded off with state space complexity*, by "unfolding" the control u_k into its m components, which are applied *one agent-at-a-time* rather than all-agents-at-once.

In particular, we can reformulate the problem by breaking down the collective decision u_k into m individual component decisions, thereby reducing the complexity of the control space while increasing the complexity of the state space. The potential advantage is that *the extra state space complexity does not affect the computational requirements of some RL algorithms, including rollout*.

To this end, we introduce a modified but equivalent problem, involving one-at-a-time agent control selection. At a state x, we break down the control u into the sequence of the m controls $u^1, u^2, \ldots, u^m$, and between x and the next state $\bar{x} = f(x, u, w)$, we introduce artificial intermediate "states" $(x, u^1), (x, u^1, u^2), \ldots, (x, u^1, \ldots, u^{m-1})$, and corresponding transitions. The choice of the last control component u^m at "state" $(x, u^1, \ldots, u^{m-1})$ marks the transition to the next state $\bar{x} = f(x, u, w)$, while incurring cost $g(x, u, w)$; see Fig. 1.6.9.

It is evident that this reformulated problem is equivalent to the origi-

nal, since any control choice that is possible in one problem is also possible in the other problem, while the cost structure of the two problems is the same. In particular, every policy $\mu = (\mu^1, \ldots, \mu^m)$ of the original problem, including a base policy in the context of rollout, is admissible for the reformulated problem, and has the same cost function for the original as well as the reformulated problem.

The motivation for the reformulated problem is that the control space is simplified at the expense of introducing $m-1$ additional layers of states, and the corresponding $m-1$ cost-to-go functions

$$J^1(x, u^1), J^2(x, u^1, u^2), \ldots, J^{m-1}(x, u^1, \ldots, u^{m-1}).$$

The increase in size of the state space does not adversely affect the operation of rollout, since the Q-factor minimization (1.77) is performed for just one state at each stage.

The major fact that can be proved about multiagent rollout (see Section 2.9 and the end-of-chapter references) is that it *achieves cost improvement*:

$$J_{\tilde{\mu}}(x) \leq J_\mu(x), \qquad \text{for all } x,$$

where $J_\mu(x)$ is the cost function of the base policy $\mu = (\mu^1, \ldots, \mu^m)$, and $J_{\tilde{\mu}}(x)$ is the cost function of the rollout policy $\tilde{\mu} = (\tilde{\mu}^1, \ldots, \tilde{\mu}^m)$, starting from state x. Furthermore, this cost improvement property can be extended to multiagent PI schemes that involve one-agent-at-a-time policy improvement operations, and have sound convergence properties. Moreover, multiagent rollout becomes the starting point for related PI schemes that are well suited for distributed operation in contexts involving multiple autonomous decision makers; see Section 2.9, the book [Ber20a], the papers [Ber20b] and [BKB20], and the tutorial survey [Ber21a].

Example 1.6.5 (Spiders and Flies)

This example is representative of a broad range of practical problems such as multirobot service systems involving delivery, maintenance and repair, search and rescue, firefighting, etc. Here there are m spiders and several flies moving on a 2-dimensional grid; cf. Fig. 1.6.10. The objective is for the spiders to catch all the flies as fast as possible.

During a stage, each fly moves to a some other position according to a given state-dependent probability distribution. Each spider learns the current state (the vector of spiders and fly locations) at the beginning of each stage, and either moves to a neighboring location or stays where it is. Thus each spider has as many as 5 choices at each stage. The control is $u = (u^1, \ldots, u^m)$, where u^ℓ is the choice of the ℓth spider, so there are about 5^m possible values of u.

To apply multiagent rollout, we need a base policy. A simple possibility is to use the policy that directs each spider to move on the path of minimum distance to the closest fly position. According to the multiagent rollout formalism, the spiders choose their moves one-at-time in the order from 1 to m,

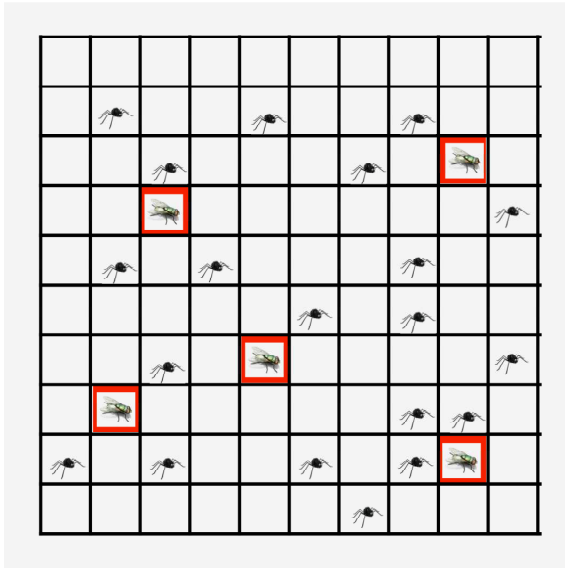

Figure 1.6.10 Illustration of a 2-dimensional spiders-and-fly problem with 20 spiders and 5 flies (cf. Example 1.6.5). The flies moves randomly, regardless of the position of the spiders. During a stage, each spider moves to a neighboring location or stays where it is, so there are 5 moves per spider (except for spiders at the edges of the grid). The total number of possible joint spiders moves is a little less than 5^{20}.

taking into account the current positions of the flies and the earlier moves of other spiders, and assuming that future moves will be chosen according to the base policy, which is a tractable computation.

In particular, at the beginning at the typical stage, spider 1 selects its best move (out of the no more than 5 possible moves), assuming the other spiders $2, \ldots, m$ will move towards their closest surviving fly during the current stage, and all spiders will move towards their closest surviving fly during the following stages, up to the time where no surviving flies remain. Spider 1 then broadcasts its selected move to all other spiders. Then spider 2 selects its move taking into account the move already chosen by spider 1, and assuming that spiders $3, \ldots, m$ will move towards their closest surviving fly during the current stage, and all spiders will move towards their closest surviving fly during the following stages, up to the time where no surviving flies remain. Spider 2 then broadcasts its choice to all other spiders. This process of one-spider-at-a-time move selection is repeated for the remaining spiders $3, \ldots, m$, marking the end of the stage.

Note that while standard rollout computes and compares 5^m Q-factors (actually a little less to take into account edge effects), multiagent rollout computes and compares ≤ 5 moves per spider, for a total of less than $5m$. Despite this tremendous computational economy, experiments with this type of spiders and flies problems have shown that multiagent rollout achieves a comparable performance to the one of standard rollout.

1.6.8 Problems with Unknown or Changing Model - Adaptive Control

Our discussion so far dealt with problems with a known mathematical model, i.e., one where the system equation, cost function, control constraints, and probability distributions of disturbances are perfectly known. The mathematical model may be available through explicit mathematical formulas and assumptions, or through computer software, such as a trained neural network, or a simulator that can emulate mathematical operations involved in the model or Monte Carlo simulation for the calculation of expected values.

From our point of view, *it makes no difference whether the mathematical model is available through closed form mathematical expressions or through a computer model, such as a simulator or a neural network*: the methods that we discuss are valid in both cases, only their suitability for a given problem may be affected by the type of model.

In practice, however, it is common that the system parameters are either not known exactly or can change over time, and this introduces potentially enormous complications.† As an example consider our oversimplified cruise control system that we noted in Example 1.3.1 or its infinite horizon version. The state evolves according to

$$x_{k+1} = x_k + bu_k + w_k, \tag{1.78}$$

where x_k is the deviation $v_k - \bar{v}$ of the vehicle's velocity v_k from the nominal $\bar{v}$, u_k is the force that propels the car forward, and w_k is the disturbance. However, the coefficient b and the distribution of w_k change frequently, and cannot be modeled with any precision because they depend on unpredictable time-varying conditions, such as the slope and condition of the road, and the weight of the car (which is affected by the number of passengers). Moreover, the nominal velocity $\bar{v}$ is set by the driver, and when it changes it may affect the parameter b in the system equation, and other parameters.‡

In this section, we will briefly review some of the most commonly used approaches for dealing with unknown models and/or parameters in optimal

† The difficulties of decision and control within a changing environment are often underestimated. Among others, they complicate the balance between offline training and on-line play, which we discussed in Section 1.1 in connection the AlphaZero. It is worth keeping in mind that as much as learning to play high quality chess is a great challenge, the rules of play are stable and do not change unpredictably in the middle of a game! Problems with changing system parameters can be far more challenging!

‡ Adaptive cruise control, which can also adapt the car's velocity based on its proximity to other cars, has been studied extensively and has been incorporated in several commercially sold car models.

control theory and practice. We should note also that unknown problem environments are an integral part of the artificial intelligence view of RL. In particular, to quote from the popular book by Sutton and Barto [SuB18], RL is viewed as "a computational approach to learning from interaction," and "learning from interaction with the environment is a foundational idea underlying nearly all theories of learning and intelligence."

The idea of learning from interaction with the environment is often connected with the idea of exploring the environment to identify its characteristics. In control theory this is often viewed as part of the *system identification* methodology, which aims to construct mathematical models of dynamic systems. The system identification process is often combined with the control process to deal with unknown or changing problem parameters, in a framework that is sometimes called *dual control*. This is one of the most challenging areas of stochastic optimal and suboptimal control, and has been studied intensively since the early 1960s.

Let us also note that modern advances in neural network and artificial intelligence technologies have expanded significantly the range of the system identification methodology. As an example, we refer to Section 2.3.8, which provides a brief discussion of *world models*. These can be used as a computational surrogate for an exact model to provide predictions of the system's trajectories beyond the first step of lookahead minimization, in contexts of approximation in value space and rollout.

Robust and Adaptive Control

Given a controller design that has been obtained assuming a nominal DP problem model, one possibility is to simply ignore changes in problem parameters. We may then try to investigate the performance of the current design for a suitable range of problem parameter values, and ensure that it is adequate for the entire range. This is sometimes called a *robust controller design*. For example, consider the oversimplified cruise control system of Eq. (1.78) with a linear controller of the form $\mu(x) = Lx$ for some scalar L. Then we check the range of parameters b for which the current controller is stable (this is the interval of values b for which $|1 + bL| < 1$), and ensure that b remains within that range during the system's operation.

The more general class of methods where the controller is modified in response to problem parameter changes is part of a broad field known as *adaptive control*, i.e., control that adapts to changing parameters. This is a rich methodology that we will discuss only briefly in this book. For the moment, let us just mention a simple time-honored adaptive control approach for a broad class of continuous-state problems without a known mathematical or computer model, called *PID (Proportional-Integral-Derivative) control*, for which we refer to the control literature, including the books by Aström and Hagglund [AsH95], [AsH06], and the end-of-chapter references on adaptive control (also Section 5.7 of the RL textbook [Ber19a]).

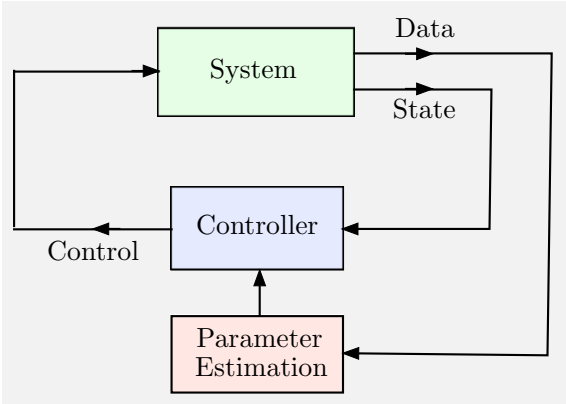

Figure 1.6.11 Schematic illustration of concurrent parameter estimation and system control. The system parameters are estimated on-line and the estimates are periodically passed on to the controller.

In particular, PID control aims to maintain the output of a single-input single-output dynamic system around a set point or to follow a given trajectory, as the system parameters change within a relatively broad range. In its simplest form, the PID controller is parametrized by three scalar parameters, which may be determined by a variety of methods, some of them manual/heuristic. PID control is used widely and with success, although its range of application is mainly restricted to single-input, single-output continuous-state control systems.

Dealing with Unknown Parameters Through System Identification

In PID control, no attempt is made to maintain a mathematical model and to track unknown model parameters as they change. An alternative and apparently reasonable form of suboptimal control is to separate the control process into two phases, a *system identification phase* and a *control phase*. In the first phase the unknown parameters are estimated, while the control takes no account of the interim results of estimation. The final parameter estimates from the first phase are then used to implement an optimal or suboptimal policy in the second phase. This alternation of estimation and control phases may be repeated several times during any system run in order to take into account subsequent changes of the parameters. Moreover, it is not necessary to introduce a hard separation between the identification and the control phases. They may be going on simultaneously, with new parameter estimates being introduced into the control process, whenever this is thought to be desirable; see Fig. 1.6.11.

One drawback of this approach is that it is not always easy to determine when to terminate one phase and start the other. A second difficulty, of a more fundamental nature, is that the control process may make some

of the unknown parameters invisible to the estimation process. This is known as the problem of *parameter identifiability*, which is discussed in the context of optimal control in several sources, including [BoV79] and [Kum83]; see also [Ber17a], Section 6.7.

Example 1.6.6 (Parameter Identifiability Under Closed-Loop Control)

For a simple example, consider the scalar system

$$x_{k+1} = ax_k + bu_k, \qquad k = 0, \ldots, N-1,$$

and the quadratic cost

$$\sum_{k=1}^{N} (x_k)^2.$$

Assuming perfect state information, if the parameters a and b are known, it can be seen that the optimal control law is

$$\mu_k^*(x_k) = -\frac{a}{b} x_k,$$

which sets all future states to 0. Assume now that the parameters a and b are unknown, and consider the two-phase method. During the first phase the control law

$$\tilde{\mu}_k(x_k) = \gamma x_k \tag{1.79}$$

is used (γ is some scalar; for example, $\gamma = -\frac{\overline{a}}{\overline{b}}$, where $\overline{a}$ and $\overline{b}$ are some a priori estimates of a and b, respectively). At the end of the first phase, the control law is changed to

$$\overline{\mu}_k(x_k) = -\frac{\hat{a}}{\hat{b}} x_k,$$

where $\hat{a}$ and $\hat{b}$ are the estimates obtained from the estimation process. However, with the control law (1.79), the closed-loop system is

$$x_{k+1} = (a + b\gamma) x_k,$$

so the estimation process can at best yield the value of $(a + b\gamma)$ but not the values of both a and b. In other words, the estimation process cannot discriminate between pairs of values (a_1, b_1) and (a_2, b_2) such that

$$a_1 + b_1\gamma = a_2 + b_2\gamma.$$

Therefore, a and b are not identifiable when feedback control of the form (1.79) is applied.

On-line parameter estimation algorithms, which address among others the issue of identifiability, have been discussed extensively in the control